Airbus Ship Detection – Project Documentation

1. Introduction

Analyzing satellite imagery and monitoring maritime traffic is a crucial area for global trade, maritime security, and environmental protection. The goal of this project is to develop and train a Deep Learning model capable of automatically detecting and segmenting ships in satellite images. The task is carried out within the framework of the "Airbus Ship Detection Challenge" hosted on Kaggle. This documentation details the technologies used, the data processing steps, the chosen neural network architecture, as well as the training and evaluation processes.

---

2. Topic Overview and Previous Solutions

The task addresses a classic Computer Vision problem: semantic segmentation. The objective is not merely to decide whether a ship exists in the image (classification) or to mark it with a bounding box (object detection), but to outline the ships with pixel-level precision.

Challenges

On satellite images, ships are often tiny compared to the vast sea surface. Furthermore, a significant portion of the images (approx. 70-80%) contains no ships at all. This class imbalance poses a serious challenge for traditional algorithms, as the model might converge to the "optimal" strategy of predicting everything as background (sea).

Previous and Alternative Solutions

Before the rise of deep learning, traditional image processing methods (e.g., edge detection, thresholding, texture analysis) were used, but these were sensitive to noise and varying lighting conditions. Modern solutions are primarily built on Convolutional Neural Networks (CNNs):

- FCN (Fully Convolutional Networks): The first step towards pixel-level predictions.
- Mask R-CNN: While effective, it is better suited for instance segmentation (separating individuals) and is computationally more expensive.
- U-Net: An architecture originally introduced for biomedical imaging that became a standard. It is excellent for precise segmentation of small objects even with limited data. We chose this architecture for the current project.

System Design: The Network Architecture (U-Net)

We employed a U-Net architecture, which is the "de facto" standard for semantic segmentation. The network has a "U" shape consisting of two main parts:

- Encoder (Downsampling path): This section is responsible for extracting image features. It consists of a series of convolutional layers followed by pooling layers. As we go deeper, spatial resolution decreases, but the number of channels (extracted abstract features) increases. This part "compresses" the visual information.
- Decoder (Upsampling path): This section restores the original image size to create the segmentation mask. It uses "Transposed Convolution" (or Upsampling) layers to increase resolution.
- Skip Connections: The most important elements of U-Net. These directly connect the outputs of the Encoder layers to the corresponding levels of the Decoder. This allows the network to preserve fine local information (e.g., the exact edges of ships) during compression, resulting in a much sharper and more accurate output.

Implementation

The implementation was done in Python using the PyTorch framework.

Data Acquisition and Preparation

The dataset was downloaded via the Kaggle API (train_ship_segmentations_v2.csv and associated images).

1. Data Cleaning: After loading the CSV file, duplicate entries were removed, and we verified that the files physically existed on the disk.
2. RLE Decoding: Masks are stored in "Run-Length Encoding" (RLE) format (efficient compression). We implemented an rle_decode function to convert this text format into a binary mask (768x768 pixels), where a value of 1 represents a ship and 0 represents the background.
3. Balancing: Since a significant part of the data consists of empty images (containing only water), balancing the training set was a critical step.
    - We collected all images containing ships (positive).
    - From the empty images (negative), we randomly sampled enough to achieve a positive:negative ratio of 1:2. This prevented the model from becoming "lazy" and optimizing for the dominant (empty) class.
4. Data Split: The resulting balanced dataset (approx. 127,000 images) was split in a stratified manner:

- Training set: 90%
- Validation set: 5%
- Test set: 5% Stratification ensures that the proportion of images with ships is identical across all sets.

Custom Dataset

We created an AirbusDataset class (inheriting from PyTorch Dataset) that dynamically loads images and generates masks.

- It handles corrupted files (error handling: returns an empty mask in case of error).
- It merges RLE codes belonging to multiple ships into a single mask.
- It allows for the application of augmentations (transformations).

Training

The training process was conducted using the following parameters and strategies:

- Loss Function: We implemented a combined BCEDiceLoss.
  - Binary Cross Entropy (BCE): Monitors the correct classification of pixels.
  - Dice Loss: Measures the overlap between the segmented area and the ground truth area (Intersection over Union type metric), which is particularly effective for handling class imbalance.
  - Weighting: 50% BCE + 50% Dice.
- Optimizer: We used the Adam optimizer with a learning rate of lr=1e-4.
- Scheduler: We applied the ReduceLROnPlateau scheduler. If the validation loss does not improve for 1 epoch, the learning rate is halved (factor 0.5). This helps the model fine-tune at the end of convergence.
- Mixed Precision: To speed up training and reduce memory usage, we used torch.amp.GradScaler (automatic mixed precision).
- Hardware: The code uses dual GPU acceleration (cuda device).

The training loop was designed for 30 epochs, with an "Early Stopping" mechanism and "Model Checkpointing" (saving the model only if the validation loss improves compared to the previous best). To test the training function and callbacks a test training is available using the same setup for less samples.

Evaluation

The model's performance was measured on the validation and test sets.

Metrics

- Training Loss: Combined error measured on the training set. Logs showed a continuous decrease (e.g., 0.0802 -> 0.0778 in the first few epochs).
- Validation Loss: Error measured on the validation set, based on which the model was saved (best value in logs: 0.0725).
- Dice Score: The most important metric in segmentation. The final Dice Score achieved on the validation set was 0.8557, meaning the predicted ship area and the real ship area overlap by approximately 85.6%. This is considered an excellent result given the difficulty of the task.
- Image-level Detection Metrics: In addition to pixel-level evaluation, image-level ship detection performance was assessed by classifying each image as either containing a ship or not. The evaluation produced counts of true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN), from which precision, recall, and accuracy were calculated. These metrics provide insight into how reliably the model detects the presence or absence of ships, complementing the Dice Score.

Visualization

During evaluation, visual checks were also performed on elements of the test set:

1. Original satellite image.
2. Real mask (Ground Truth).
3. Model predicted mask (Prediction). This allows for qualitative analysis (e.g., how well it recognizes ships near the coast or small boats).

Summary

We successfully implemented and trained a U-Net based deep learning model for ship detection. As a result of data balancing (1:2 ratio), a modern loss function (BCE+Dice), and an optimized training strategy (LR scheduling, Mixed Precision), the model achieved a high Dice score exceeding 0.85.